

Bazele Inteligenței Artificiale

Lab 2. Modele liniare

Efrem Dragoș-Sebastian-Mihaly

Tipuri de modele în ML

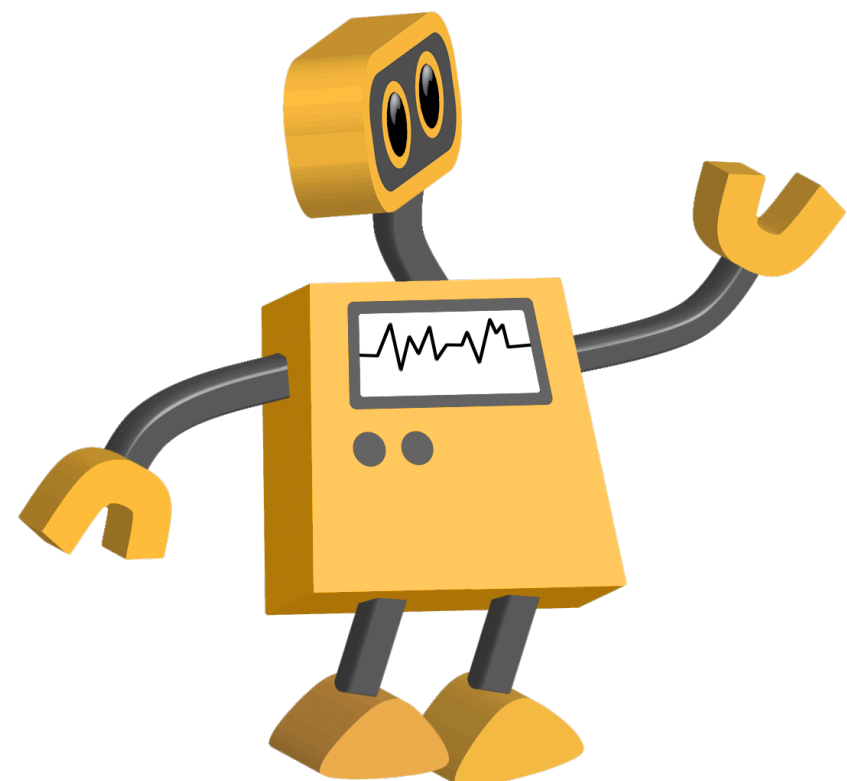
- În general, sunt două tipuri mari și late:

Supervizate

Nesupervizate

Supervizate

- Învăță din date care au deja etichete (adică știm răspunsul corect pentru fiecare element utilizat pentru a antrena modelul)

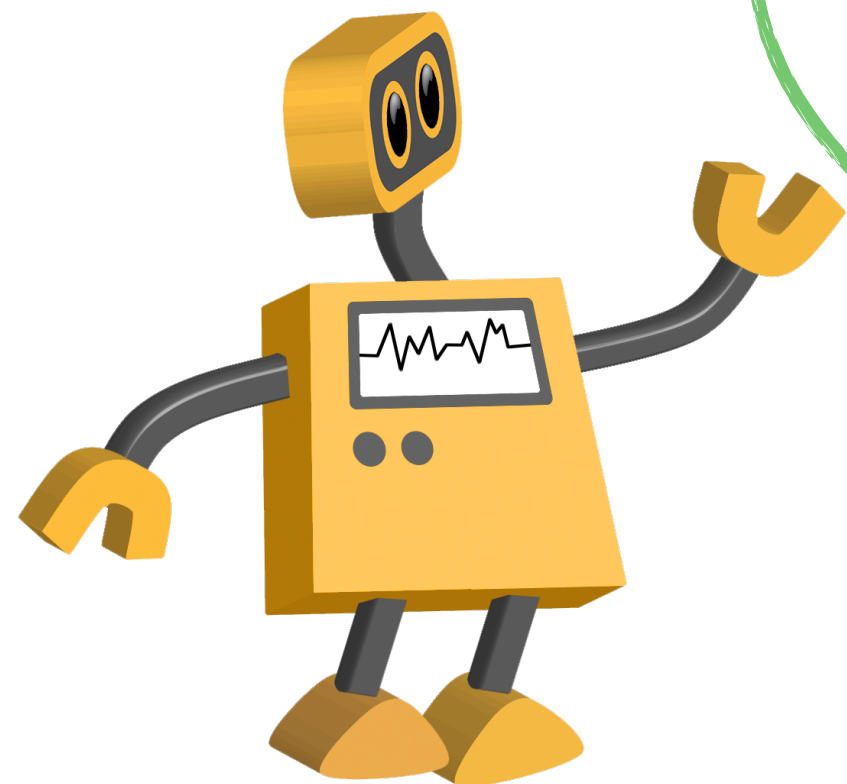


(Modelul vostru super fain)

Supervizate

- Pentru a antrena modele de ML, avem nevoie de caracteristici / attribute / features.
- Exemplu:
 - Dacă încercăm să prezicem prețul unui apartament, caracteristicile ar putea fi:

Suprafața în metrii pătrați
Numărul de camere
Vârsta clădirii



(Modelul vostru super fain)



Crearea unui set de date

Suprafața în metri pătrați

Numărul de camere

Vârsta clădirii


$$x_1 = (50 \quad 2 \quad 5)$$

$$x_2 = (60 \quad 2 \quad 10)$$

$$x_3 = (70 \quad 3 \quad 15)$$

Crearea unui set de date

Suprafața în metrii pătrați

Numărul de camere

Vârsta clădirii

$$x_1 = (50 \quad 2 \quad 5)$$

$$x_2 = (60 \quad 2 \quad 10)$$

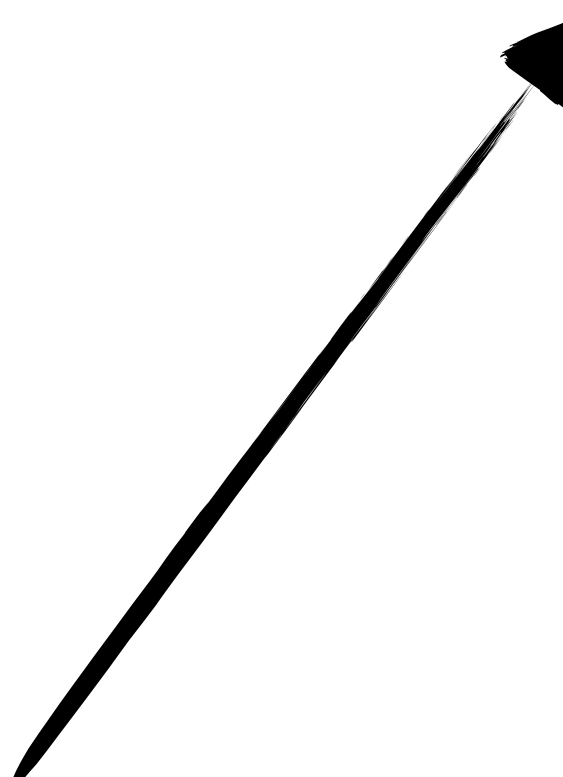
$$x_3 = (70 \quad 3 \quad 15)$$

$$y_1 = 60000$$

$$y_2 = 75000$$

$$y_3 = 90000$$

leșirea modelului (ce încercăm să prezicem, în acest caz prețul în euro)



Crearea unui set de date

$$\begin{array}{lll} x_1 = (50 & 2 & 5) & x_2 = (60 & 2 & 10) & x_3 = (70 & 3 & 15) \\ y_1 = 60000 & y_2 = 75000 & y_3 = 90000 \end{array}$$

$$X = \begin{pmatrix} 50 & 2 & 5 \\ 60 & 2 & 10 \\ 70 & 3 & 15 \end{pmatrix} \quad y = \begin{pmatrix} 60000 \\ 75000 \\ 90000 \end{pmatrix}$$

Supervizate

- Pentru a antrena modele de ML, avem nevoie de caracteristici / attribute / features.
- Exemplu:
 - Dacă încercăm să prezicem prețul unui apartament, caracteristicile ar putea fi:

Suprafața în metrii pătrați
Numărul de camere
Vârsta clădirii



Supervizate

variabila dependentă

Suprafața în metrii pătrați

50

60

70

80

90

100

110

120

variabila independentă

prețul unui apartament

35000

40000

48000

55000

62000

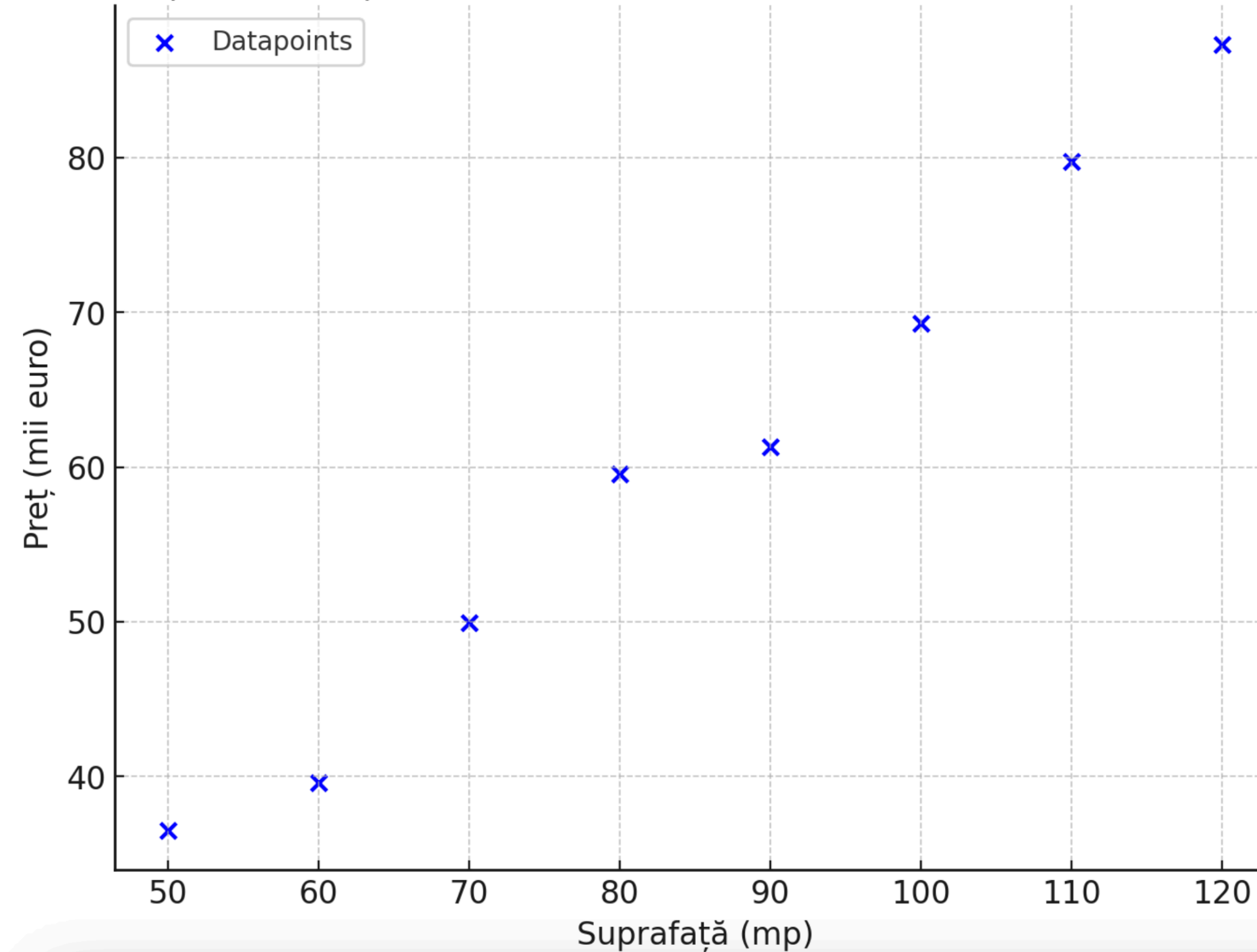
70000

75000

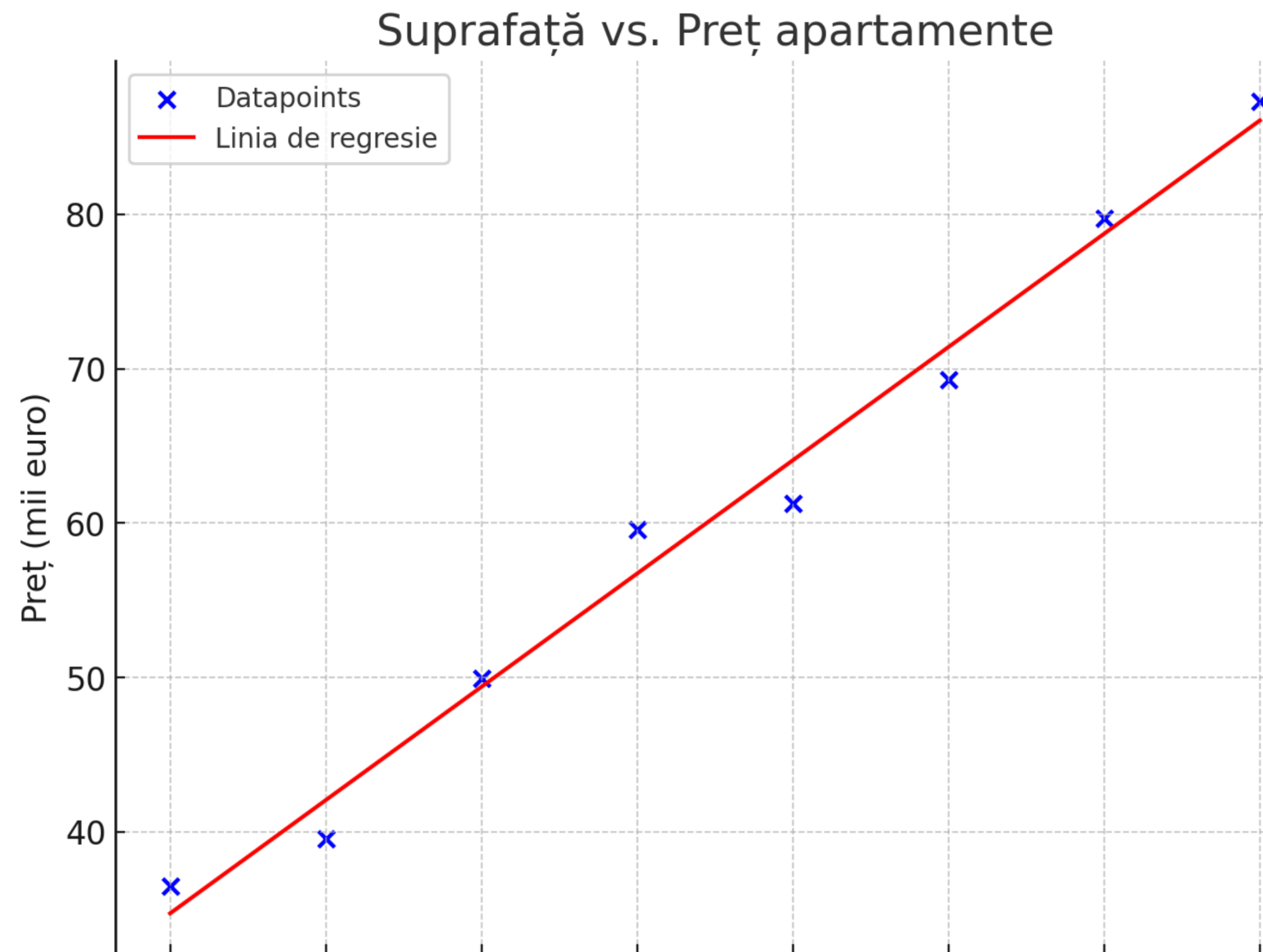
85000

Găsirea modelului

Suprafață vs. Preț apartamente (Datapoints, fără linia de regresie)

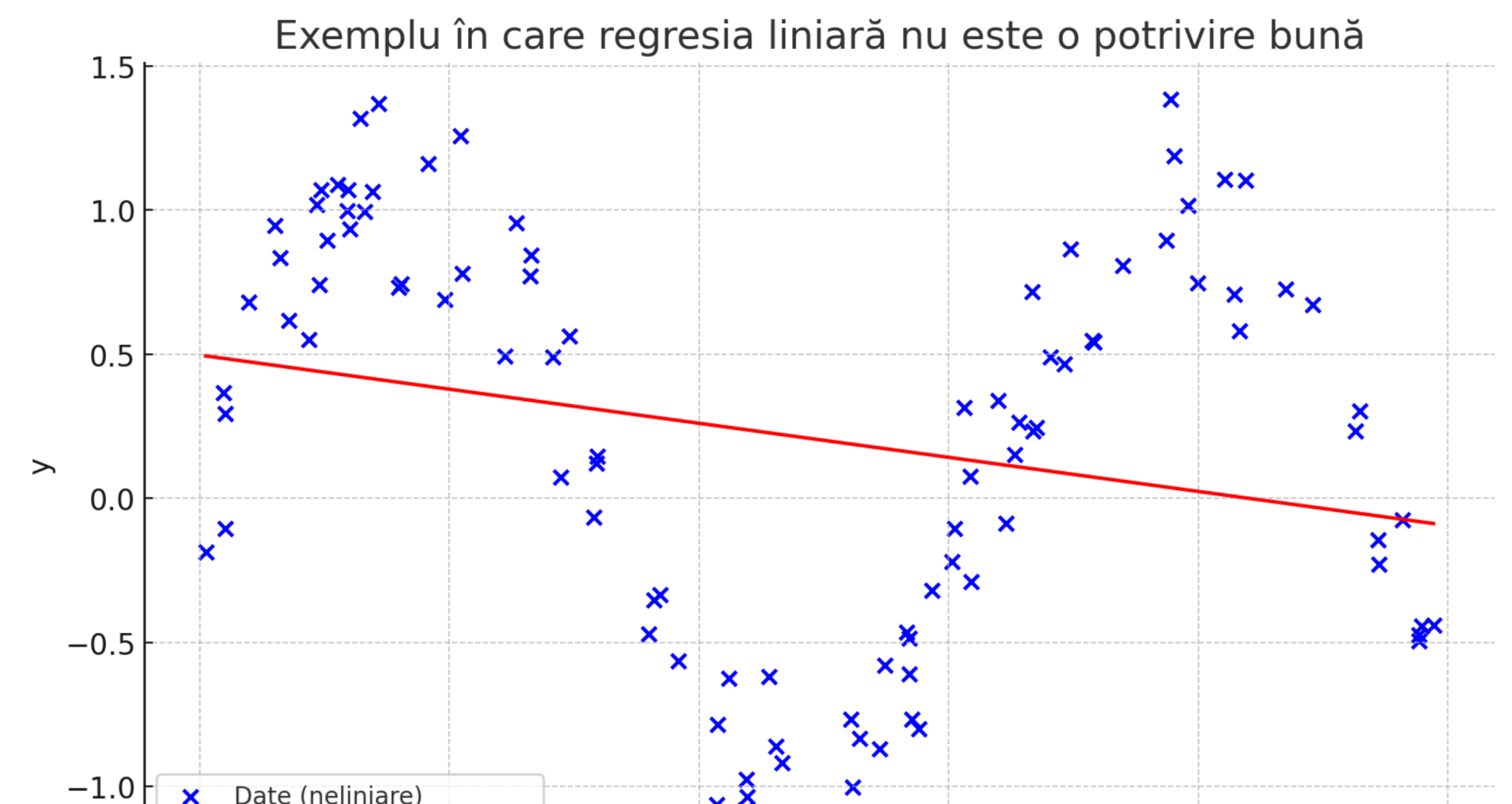


Găsirea modelului



Regresia liniară

- Scopul regresiei liniare este de a găsi o linie dreaptă (sau un plan în cazul mai multor variabile) care aproximează cel mai bine datele noastre.
- Regresia liniară funcționează bine atunci când există o relație liniară între variabila dependentă (cea pe care vrei să o prezici) și variabila independentă (cea pe care o folosești pentru predicție). Adică, atunci când o creștere a uneia duce la o creștere/scădere constantă a celeilalte.



Regresia liniară

- Ecuația liniei obținute de regresia liniară are forma: $y = m \cdot X + b$
 - y este valoarea pe care o prezicem (prețul unui apartament),
 - X este variabila pe care o folosim pentru a prezice (suprafața în metri pătrați),
 - m este panta (cât de abruptă e linia, adică cum se schimbă y când X crește),
 - b este interceptul, adică punctul unde linia intersectează axa Y .

Regresia liniară

$$y = \underbrace{m}_{w_1 \text{ panta}} \cdot X + \underbrace{b}_{w_0 \text{ intercept}}$$

$$X = \begin{pmatrix} 50 \\ 70 \\ 80 \\ 90 \\ 100 \\ 110 \\ 120 \end{pmatrix}$$

$$y = \begin{pmatrix} 35000 \\ 40000 \\ 48000 \\ 55000 \\ 62000 \\ 70000 \\ 75000 \\ 85000 \end{pmatrix}$$

Forma generală:

$$\underset{\substack{\uparrow \\ \text{predicted} \\ \text{value}}}{\hat{y}} = \underset{\substack{\uparrow \\ \text{bias}}}{w_0} + \underset{\substack{\uparrow \\ \text{feature} \\ \text{weight}}}{w_1} \overset{\substack{\downarrow \\ \text{feature}}}{x_1} + \dots + \underset{\substack{\uparrow \\ \text{feature} \\ \text{weight}}}{w_d} \overset{\substack{\downarrow \\ \text{feature}}}{x_d}.$$

Regresia liniară

```
[11] import numpy as np
      from sklearn.linear_model import LinearRegression
      from sklearn.metrics import mean_squared_error

      X = np.array([50, 60, 70, 80, 90, 100, 110, 120]).reshape(-1, 1) # Suprafata
      y = np.array([35, 40, 48, 55, 62, 70, 75, 85]) # Pretul in mii de euro

      model = LinearRegression()
      model.fit(X, y)

      y_pred = model.predict(X)

      coeficient = model.coef_
      intercept = model.intercept_

      coeficient, intercept, mean_squared_error(y_pred, y), y[2], y_pred[2]
```

```
⇒ (array([0.71190476]),
   -1.7619047619047734,
   0.8630952380952381,
   48,
   48.07142857142857)
```

Cum a reușit?

Cum a reușit?

Gradient Descent

- Un model de ML învață prin minimizarea unei funcții de cost în raport cu parametri modelului;
- Funcția de cost este diferența dintre ce dă modelul ca ieșire față de ce ar trebui să dea;
- Nu există o funcție de cost standard pentru a rezolva o problemă, e nevoie de experimente;

Funcția de cost

Suprafața în metri pătrați

prețul unui apartament

50

35000

60

40000

70

48000

80

55000

90

62000

100

70000

110

75000

120

85000

Funcția de cost

Suprafața în metrii pătrați	prețul unui apartament	prețul unui apartament (prezis)
50	35000	36850
60	40000	40050
70	48000	49128
80	55000	54998
90	62000	61999
100	70000	69988
110	75000	77000
120	85000	85002

Funcția de cost

prețul unui apartament		prețul unui apartament (prezis)
$\frac{1}{8} [$		$36850)^2 +$
$(35000$	-	
$(40000$	-	$40050)^2 +$
$(48000$	-	$49128)^2 +$
$(55000$	-	$54998)^2 +$
$(62000$	-	$61999)^2 +$
$(70000$	-	$69988)^2 +$
$(75000$	-	$77000)^2 +$
$(85000$	-	$85002)^2] = \text{cost}$

Gradient Descent

Funcția de cost

$$\text{Eroare} = \frac{1}{n} \sum_{i=1}^n \left(Y_i - \hat{Y}_i \right)^2$$

model(X)

Parametri
modelului

$$m_{\text{nou}} = m_{\text{vechi}} - \alpha \cdot \frac{\partial}{\partial m} \text{Eroare}$$

$$b_{\text{nou}} = b_{\text{vechi}} - \alpha \cdot \frac{\partial}{\partial b} \text{Eroare}$$

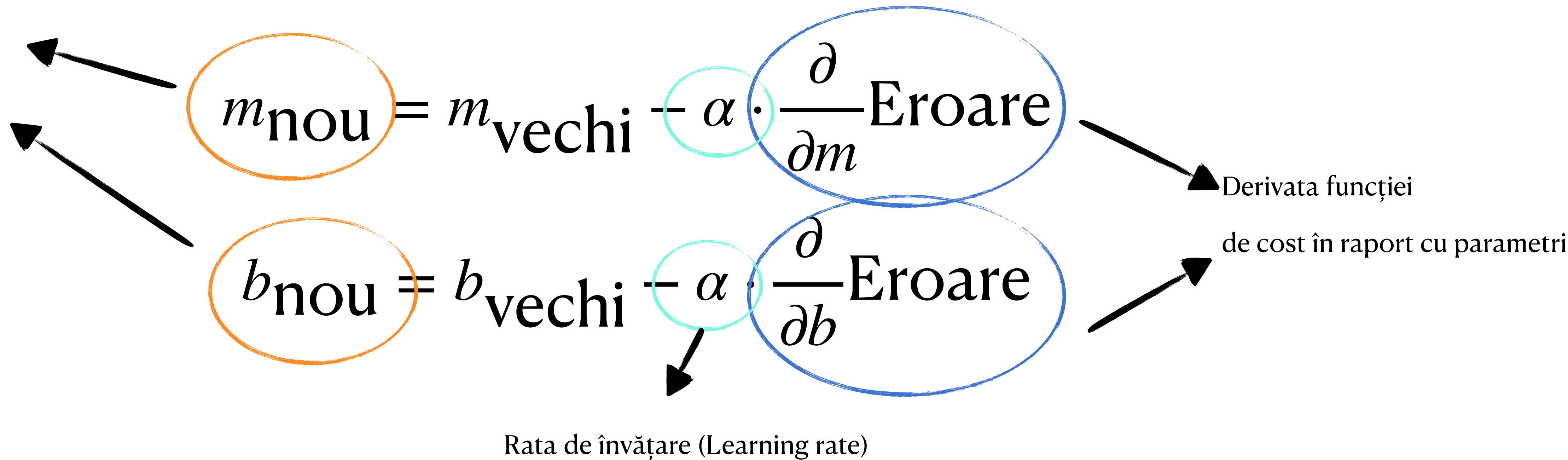
Derivata funcției

de cost în raport cu parametri

Rata de învățare (Learning rate)

Gradient Descent

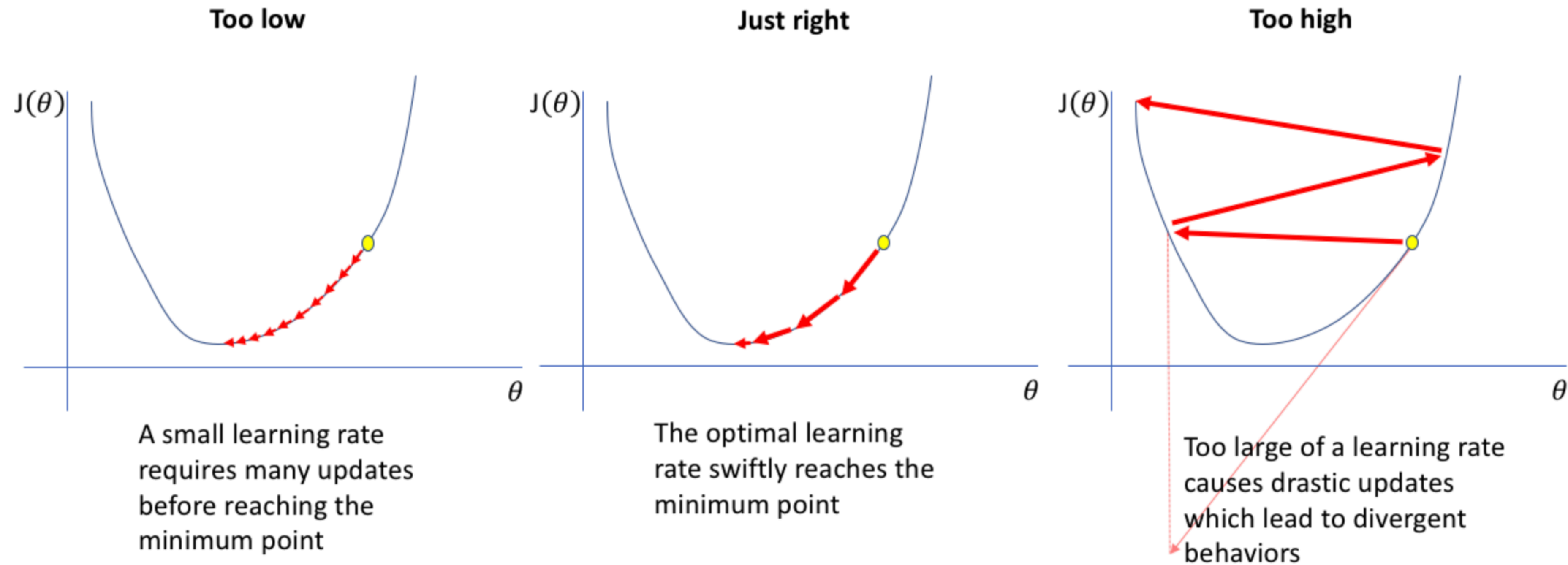
Parametri
modelului



Parametri care nu sunt actualizați de către gradient descent (cum este alpha) poartă numele de hiperparametri.

Formula generală: $w_{\text{nou}} = w_{\text{vechi}} - \alpha \cdot \frac{\partial}{\partial w} \text{Eroare}$ pentru toți w

Gradient Descent



Mai multe tipuri de Gradient Descent

```
eta = 0.2 # learning rate
n_epochs = 1000
n = 100
```

Batch

```
w = np.random.randn(2,1) # random initialization
```

```
for iteration in range(n_epochs):
    gradients = 1/n * X_1.T.dot(X_1.dot(w) - y)
    w = w - eta * gradients
```

w

```
array([[4.21509616],
       [2.77011339]])
```

```
eta = 0.2
n_epochs = 1000
n = 100 # size of the dataset
```

Stochastic

```
w = np.random.randn(2,1) # random initialization
```

```
for epoch in range(n_epochs):
    for i in range(n):
        random_index = np.random.randint(n)
        xi = X_1[random_index:random_index+1]
        yi = y[random_index:random_index+1]
        gradients = xi.T.dot(xi.dot(w) - yi)
        w = w - eta * gradients
```

w

```
array([[4.17686315],
       [2.85986577]])
```

```
eta = 0.2
n_epochs = 1000
batch_size = 20
```

Mini-batch

```
w = np.random.randn(2,1) # random initialization
```

```
for epoch in range(n_epochs):
    shuffled_indices = np.random.permutation(n)
    X_1_shuffled = X_1[shuffled_indices]
    y_shuffled = y[shuffled_indices]
    for i in range(0, n, batch_size):
        xi = X_1_shuffled[i:i+batch_size]
        yi = y_shuffled[i:i+batch_size]
        gradients = 1/batch_size * xi.T.dot(xi.dot(w) - yi)
        w = w - eta * gradients
```

w

```
array([[4.19811521],
       [2.75356112]])
```

Epocă: o trecere prin tot setul de antrenare

Batch size: În cazul mini-batch GD, reprezintă numărul de date folosite în actualizarea parametrilor modelului

Regresia Liniară cu SGDRegressor

- Clasa LinearRegression() în Scikit-Learn, de fapt, nu se bazează pe Gradient Descent, însă s-a creat o clasă care folosește Stochastic Gradient Descent

```
import numpy as np
from sklearn.linear_model import SGDRegressor
from sklearn.metrics import mean_squared_error

X = np.array([50, 60, 70, 80, 90, 100, 110, 120]).reshape(-1, 1) # Suprafata
y = np.array([35, 40, 48, 55, 62, 70, 75, 85]) # Pretul in mii de euro

model = SGDRegressor(max_iter=1000, tol=1e-3, eta0=0.0001, learning_rate='constant', random_state=42)
# 'eta0' is the learning rate, 'learning_rate' controls how it changes (constant in this case)
# tol is the tolerance for stopping; when the improvement in the loss is smaller than tol, the training stops early

model.fit(X, y)

y_pred = model.predict(X)

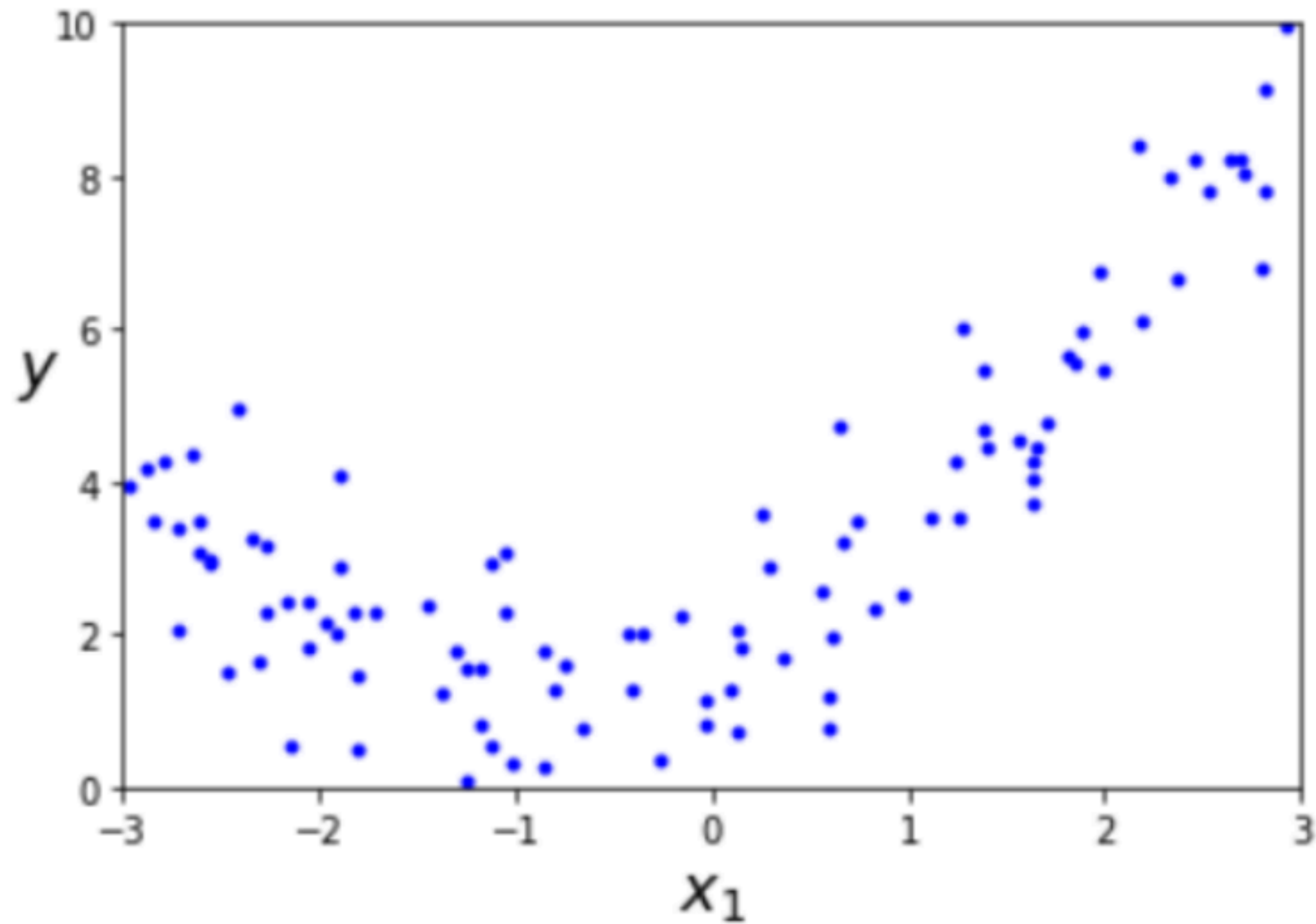
coefficient = model.coef_
intercept = model.intercept_

coefficient, intercept, mean_squared_error(y_pred, y), y[2], y_pred[2]
```

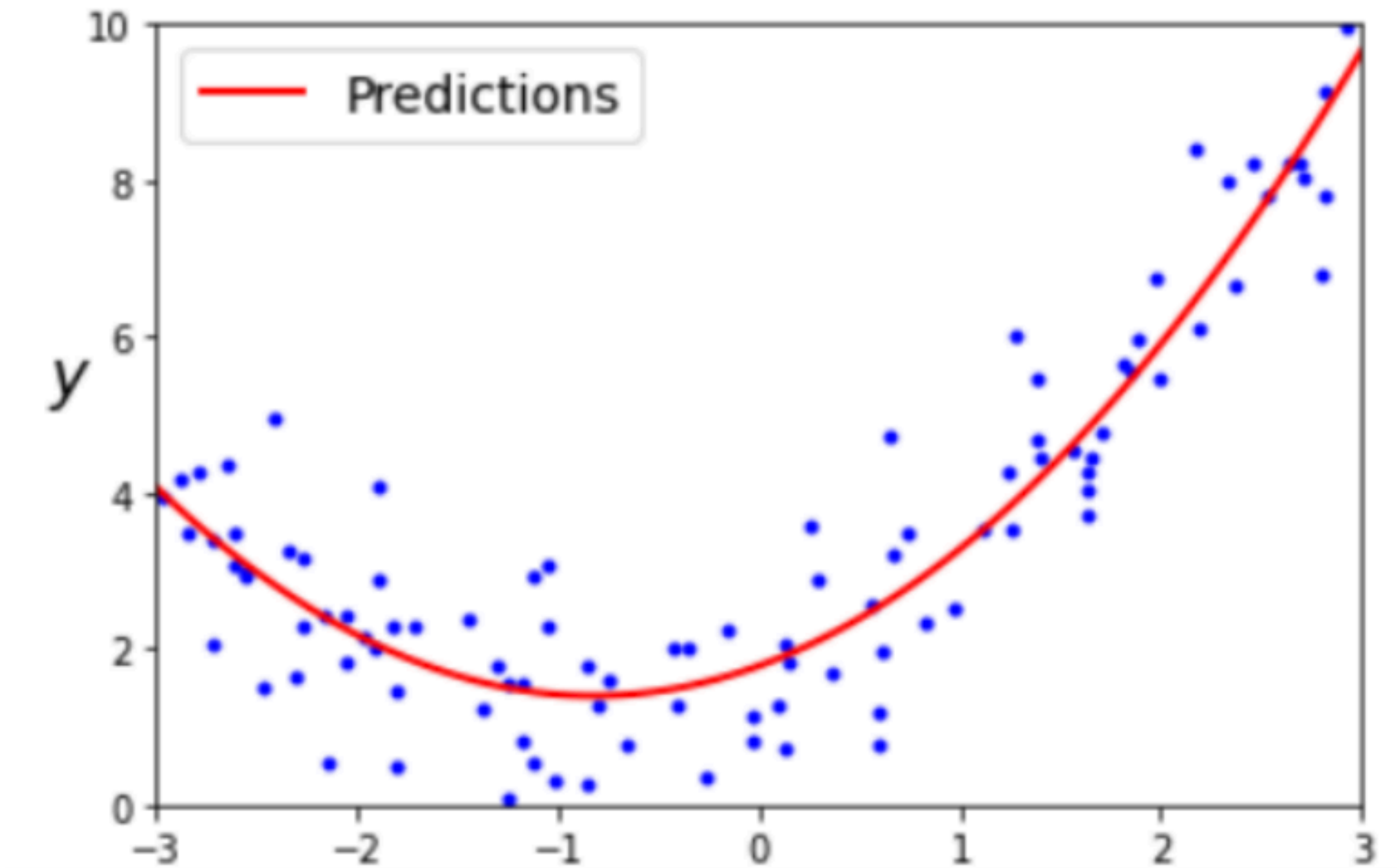
```
(array([0.68721894]),
 array([0.00761519]),
 1.2911179940057527,
 48,
 48.11294084760684)
```

- Util când antrenăm pe seturi de date mari

Regresia Polinomială



Regresia Polinomială



Regresia Polinomială

```
import numpy as np
from sklearn.linear_model import SGDRegressor
from sklearn.metrics import mean_squared_error
from sklearn.preprocessing import PolynomialFeatures

np.random.seed(42)

# Date de antrenare: 100 de date, caracteristica X este random iar y e de forma:  $1/2 * X^2 + X + 2 + \text{zgomot}$ 
n = 100
X = 6 * np.random.rand(n, 1) - 3
y = 0.5 * X**2 + X + 2 + np.random.randn(n,1)

# Transformare caracteristici
poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)

# Definire model
model = SGDRegressor(max_iter=1000, tol=1e-3, eta0=0.01, learning_rate='constant', random_state=42)

# Antrenare model
model.fit(X_poly, y.ravel())

# Verificare model pe setul de antrenare
y_pred = model.predict(X_poly)

# Obținere coeficienti si intercept
coefficient = model.coef_
intercept = model.intercept_

coefficient, intercept, mean_squared_error(y_pred, y), y[2], y_pred[2]
```

```
(array([0.93864877, 0.8362468 , 0.55403273]),
 array([0.93972708]),
 0.8134348549968392,
 array([4.45250583]),
 4.115874213875667)
```

Regresia Polinomială

```
import numpy as np
from sklearn.linear_model import SGDRegressor
from sklearn.metrics import mean_squared_error
from sklearn.preprocessing import PolynomialFeatures

np.random.seed(42)

# Date de antrenare: 100 de date, caracteristica X este random iar y e de forma:  $1/2 * X^2 + X + 2 + \text{zgomot}$ 
n = 100
X = 6 * np.random.rand(n, 1) - 3
y = 0.5 * X**2 + X + 2 + np.random.randn(n,1)

# Transformare caracteristici
poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)

# Definire model
model = SGDRegressor(max_iter=1000, tol=1e-3, eta0=0.01, learning_rate='constant', random_state=42)

# Antrenare model
model.fit(X_poly, y.ravel())

# Verificare model pe setul de antrenare
y_pred = model.predict(X_poly)

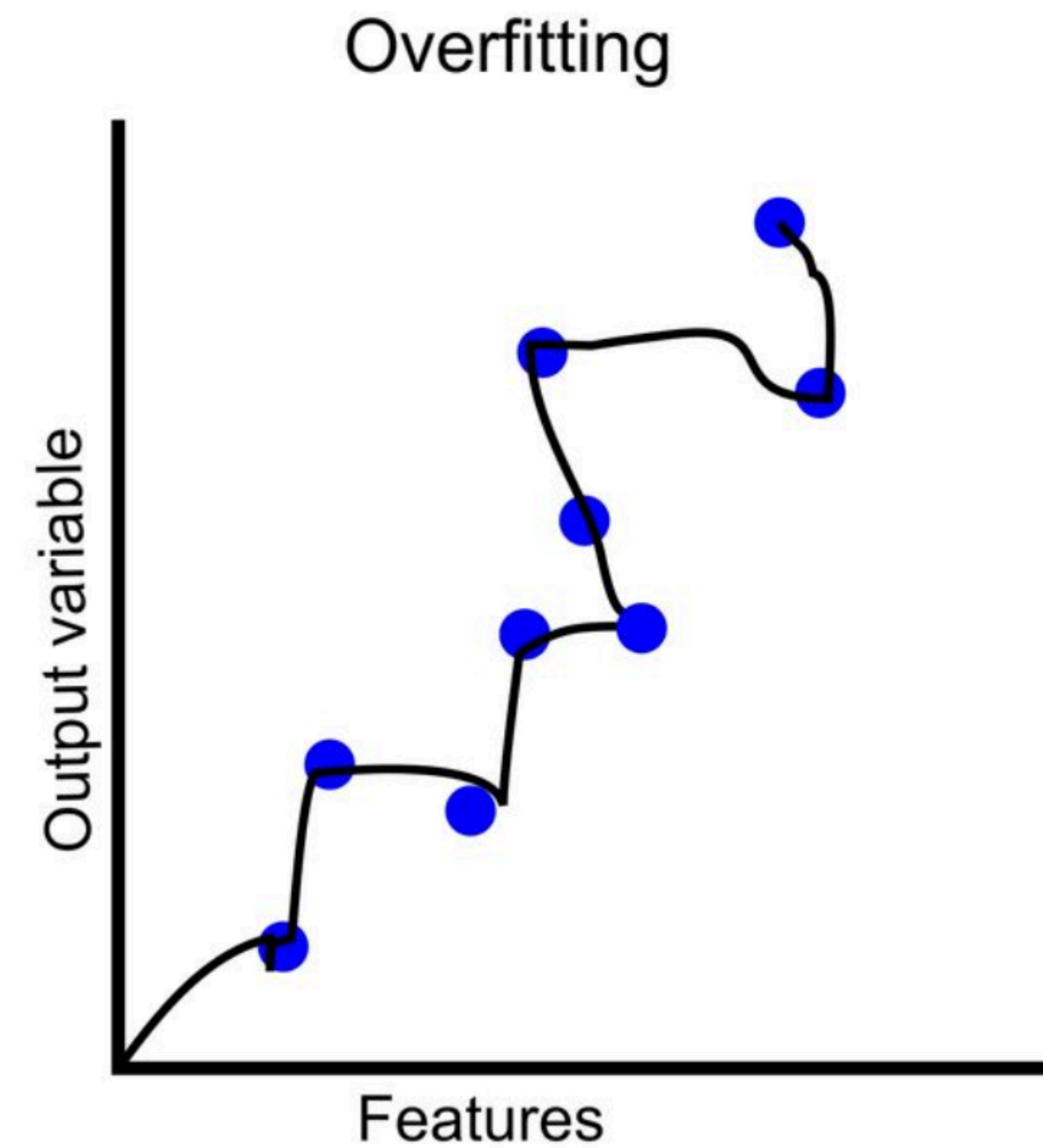
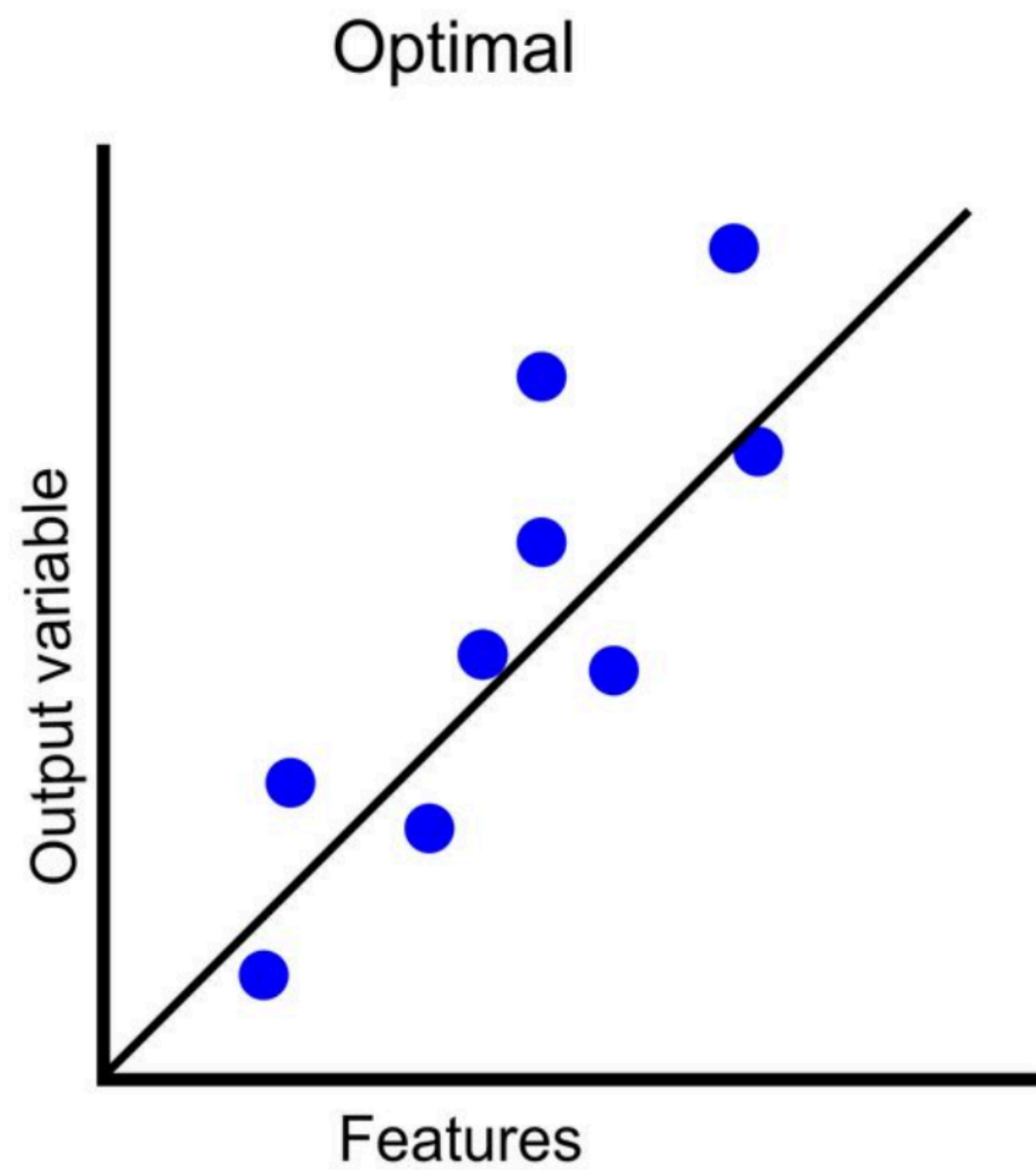
# Obținere coeficienti si intercept
coefficient = model.coef_
intercept = model.intercept_

coefficient, intercept, mean_squared_error(y_pred, y), y[2], y_pred[2]
```

```
(array([0.93864877, 0.8362468 , 0.55403273]),
 array([0.93972708]),
 0.8134348549968392,
 array([4.45250583]),
 4.115874213875667)
```

PolynomialFeatures transformă caracteristicile noastre în noi caracteristici, adăugând puteri și combinații ale acestora. Ne ajută să creăm un model care poate captura relații mai complexe, cum ar fi relații neliniare.

Overfitting și combaterea lui



Apare când modelul încearcă să învețe prea „pe de rost” datele, fără să știe să generalizeze.

Overfitting și combaterea lui

Ridge Regression

$$\text{Eroare} = \frac{1}{n} \sum_{i=1}^n \left(Y_i - \hat{Y}_i \right)^2$$

$$\text{Eroare}_{\text{ridge}} = \text{Eroare} + \lambda \sum_{j=1}^p w_j^2$$

Ridge penalizează coeficienții mari (prin adăugarea penalizării L2), forțându-i să fie mai mici. Asta face modelul mai „general”, deci mai capabil să se adapteze la date noi. Coeficienții foarte mari sugerează că modelul este prea sensibil la anumite caracteristici. Prin aplicarea penalizării L2, coeficienții nu devin zero, ci sunt micșorați, ceea ce duce la un model mai stabil.

```
import numpy as np
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_squared_error
from sklearn.preprocessing import PolynomialFeatures

np.random.seed(42)

# Date de antrenare: 100 de date, caracteristica X este random iar y e de forma: 1/2 * X^2 + X + 2 + zgomot
n = 100
X = 6 * np.random.rand(n, 1) - 3
y = 0.5 * X**2 + X + 2 + np.random.randn(n, 1)

# Transformare caracteristici
poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)

# Definire model Ridge Regression
model = Ridge(alpha=1.0) # alpha controleaza penalizarea L2

# Antrenare model
model.fit(X_poly, y.ravel())

# Verificare model pe setul de antrenare
y_pred = model.predict(X_poly)

# Obținere coeficienti si intercept
coefficient = model.coef_
intercept = model.intercept_

coefficient, intercept, mean_squared_error(y_pred, y), y[2], y_pred[2]
```

```
(array([0.          , 0.9305576 , 0.56355288]),
 1.7840063763716716,
 0.7772283095922269,
 array([4.45250583]),
 4.171227827023836)
```

Overfitting și combaterea lui

Lasso Regression

$$\text{Eroare} = \frac{1}{n} \sum_{i=1}^n \left(Y_i - \hat{Y}_i \right)^2$$

$$\text{Eroare}_{\text{lasso}} = \text{Eroare} + \lambda \sum_{j=1}^p |w_j|$$

Lasso este util atunci când avem multe caracteristici, dar doar câteva sunt cu adevărat relevante pentru predicții. Elimină caracteristicile irelevante, ajutând atât la interpretarea mai ușoară, cât și la performanțe mai bune pe date noi. Practic, face „feature selection”.

```
import numpy as np
from sklearn.linear_model import Lasso
from sklearn.metrics import mean_squared_error
from sklearn.preprocessing import PolynomialFeatures

np.random.seed(42)

# Date de antrenare: 100 de date, caracteristica X este random iar y e de forma: 1/2 * X^2 + X + 2 + zgomot
n = 100
X = 6 * np.random.rand(n, 1) - 3
y = 0.5 * X**2 + X + 2 + np.random.randn(n, 1)

# Transformare caracteristici
poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)

# Definire model Ridge Regression
model = Lasso(alpha=0.1) # alpha controleaza penalizarea L1

# Antrenare model
model.fit(X_poly, y.ravel())

# Verificare model pe setul de antrenare
y_pred = model.predict(X_poly)

# Obținere coeficienti si intercept
coeficient = model.coef_
intercept = model.intercept_

coeficient, intercept, mean_squared_error(y_pred, y), y[2], y_pred[2]
```

```
(array([0.          , 0.89934151, 0.54811812]),
 1.8275987667771458,
 0.7822708728470147,
 array([4.45250583]),
 4.14146274769837)
```

Overfitting și combaterea lui

Elastic Net Regression

$$\text{Eroare} = \frac{1}{n} \sum_{i=1}^n \left(Y_i - \hat{Y}_i \right)^2$$

$$\text{Eroare}_{\text{elastic}} = \text{Eroare} + \lambda \cdot r \cdot \sum_{j=1}^p |w_j| + \lambda \cdot (1 - r) \cdot \sum_{j=1}^p w_j^2$$

În unele situații, doar Ridge sau doar Lasso poate să nu fie suficient. Elastic Net combină penalizările L1 și L2 pentru a obține atât selecția caracteristicilor, cât și regularizarea.

```
import numpy as np
from sklearn.linear_model import ElasticNet
from sklearn.metrics import mean_squared_error
from sklearn.preprocessing import PolynomialFeatures

np.random.seed(42)

# Date de antrenare: 100 de date, caracteristica X este random iar y e de forma: 1/2 * X^2 + X + 2 + zgomot
n = 100
X = 6 * np.random.rand(n, 1) - 3
y = 0.5 * X**2 + X + 2 + np.random.randn(n, 1)

# Transformare caracteristici
poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)

# Definire model Lasso Regression
model = ElasticNet(alpha=0.1, l1_ratio=0.5) # alpha controleaza penalizarea, l1_ratio stabileste echilibrul L1 si L2

# Antrenare model
model.fit(X_poly, y.ravel())

# Verificare model pe setul de antrenare
y_pred = model.predict(X_poly)

# Obținere coeficienti si intercept
coefficient = model.coef_
intercept = model.intercept_

coefficient, intercept, mean_squared_error(y_pred, y), y[2], y_pred[2]
```

(array([0.1, 0.90142924, 0.55141065]), 1.8174818180336576, 0.7812789590887796, array([4.45250583]), 4.140631319029424)

Regresia Logistică

- Regresia logistică este o metodă folosită pentru a face predicții atunci când rezultatul are doar două opțiuni (clasificare binară). De exemplu:
 - „Va trece studentul acest examen?”
 - „Este acest email spam sau nu?”
 - „Este fain laboratorul so far sau nu?”

Regresia logistică prezice probabilități, adică șansa ca un anumit lucru să se întâmple.

Regresia Logistică în Scikit-Learn

Ex: Vom clasifica dacă un anumit exemplu din setul de date Iris este de tip Setosa (clasa 0) sau nu (clasele 1 și 2)

```
from sklearn import datasets
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

iris = datasets.load_iris()
X = iris.data
y = iris.target
y_binary = (y == 0).astype(int)
X_train, X_test, y_train, y_test = train_test_split(X, y_binary,
                                                    test_size=0.3, random_state=42)
log_reg = LogisticRegression()

log_reg.fit(X_train, y_train)

y_pred = log_reg.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

accuracy
```

Încărcarea setului de date în
memorie

Obținere caracteristici (X) și
eticheta (y)

Marcare clasa Setosa ca True
și celelalte ca False

Regresia Logistică în Scikit-Learn

Ex: Vom clasifica dacă un anumit exemplu din setul de date Iris este de tip Setosa (clasa 0) sau nu (clasele 1 și 2)

```
X_train, X_test, y_train, y_test = train_test_split(X, y_binary,  
test_size=0.3, random_state=42)
```

train_test_split împarte setul de date original (X și y) într-un set de antrenare și unul de testare.

De exemplu, aici, setul de testare va fi 30% din setul original, iar cel de antrenare va fi 70%.

random_state ne ajută la reproductibilitatea codului. Astfel, când rulăm codul de mai multe ori, obținem aceleași rezultate

Regresia Logistică în Scikit-Learn

Ex: Vom clasifica dacă un anumit exemplu din setul de date Iris este de tip Setosa (clasa 0) sau nu (clasele 1 și 2)

```
from sklearn import datasets
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

iris = datasets.load_iris()
X = iris.data
y = iris.target
y_binary = (y == 0).astype(int)
X_train, X_test, y_train, y_test = train_test_split(X, y_binary,
                                                    test_size=0.3, random_state=42)
log_reg = LogisticRegression()
log_reg.fit(X_train, y_train)
y_pred = log_reg.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
```

Încărcarea setului de date în memorie

Obținere caracteristici (X) și eticheta (y)

Marcare clasa Setosa ca True și celelalte ca False

Antrenarea modelului

Inferența pe setul de test

Obținere acuratețe

accuracy

Regresia Softmax

- Regresia logistică se folosește pentru a prezice între două opțiuni (da/nu, 0/1).
Totuși, unele probleme au mai multe posibile rezultate.
- Exemplu:
 - Avem trei tipuri de flori: Setosa, Versicolor, și Virginica.
 - Vrem să prezicem ce tip de floare avem, nu doar dacă este un anumit tip sau nu.

Regresia softmax este o extensie a regresiei logistice care se ocupă de astfel de situații, unde sunt mai multe clase posibile, nu doar două.

Regresia Softmax

```
from sklearn import datasets
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import numpy as np

iris = datasets.load_iris()

X = iris.data # caratteristici
y = iris.target # etichete

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
log_reg_softmax = LogisticRegression(multi_class='multinomial', solver='lbfgs', C = 8, max_iter=200)
log_reg_softmax.fit(X_train, y_train)

y_pred = log_reg_softmax.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

accuracy, y_pred[2], y_test[2], log_reg_softmax.predict_proba([X_test[2]]), np.argmax(log_reg_softmax.predict_proba([X_test[2]]))
```

→ /usr/local/lib/python3.10/dist-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in versi
warnings.warn(
(1.0, 2, 2, array([[1.71500624e-13, 1.79072478e-05, 9.99982093e-01]]), 2)

Regresia Softmax

Constanta de regularizare
(valoare mai mică înseamnă
regularizare mai mare)

```
log_reg_softmax = LogisticRegression(multi_class='multinomial', solver='lbfgs', C = 8, max_iter=200)
```

tipul de clasificare

(Dacă e nesetat avem regresie logistica, dacă îl
setăm cu „multinomial” avem regresie softmax)

Algoritmul folosit behind the
scenes pentru optimizare

numărul maxim de iterații pe care
solver-ul le poate parcurge până
converge (găsește o soluție optimă).

Regresia Softmax

```
▶ from sklearn import datasets
  from sklearn.linear_model import LogisticRegression
  from sklearn.model_selection import train_test_split
  from sklearn.metrics import accuracy_score
  import numpy as np

iris = datasets.load_iris()

X = iris.data # caratteristici
y = iris.target # etichete

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

log_reg_softmax = LogisticRegression(multi_class='multinomial', solver='lbfgs', C = 8, max_iter=200)

log_reg_softmax.fit(X_train, y_train)

y_pred = log_reg_softmax.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

accuracy, y_pred[2], y_test[2], log_reg_softmax.predict_proba([X_test[2]]), np.argmax(log_reg_softmax.predict_proba([X_test[2]]))

⇒ /usr/local/lib/python3.10/dist-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in versi
  warnings.warn(
(1.0, 2, 2, array([[1.71500624e-13, 1.79072478e-05, 9.99982093e-01]]), 2)
```

That's it for today!
Now let's do some exercises.

That's it for today!

Now let's do some exercises.

Laboratory 2 – Exercises

1. Use the following dataset to generate a regression dataset:

```
X = (2 - 3*np.random.normal(0,1,200)).reshape(200,1)
```

```
y = 0.5*X^3 - 2*X^2 + X + np.random.normal(-3,3,200).reshape(-1,1).
```

Apply polynomial regression algorithm and plot the predictions of the model

```
for X_test = np.linspace(-5, 10, 200).reshape(200, 1).
```

2. Use the following dataset to generate a regression dataset:

```
X = 3*np.random.rand(150, 1)
```

```
y = -2 + 3*X + np.random.randn(150, 1).
```

Perform ridge regression using mini-batch gradient descent. Use a learning rate of 0.06, 15 epochs and a batch size of 10. Set λ to 0.01. Plot the prediction of the model and print the model weights.

3. Use the *digits* dataset in order to apply the softmax regression algorithm.

Load the dataset by calling the `load_digits` function. Split the data such that the test split is 35% of the initial dataset. Print the shapes of training and testing dataset. Train the algorithm for 700 epochs, use the "lbfgs" solver and set the hyperparameter C to 5. After the model is trained, print the test set accuracy. Also predict and print the probabilities that the model assigned to the ten classes for the second example in the test set.